

Language Agent Tree Search

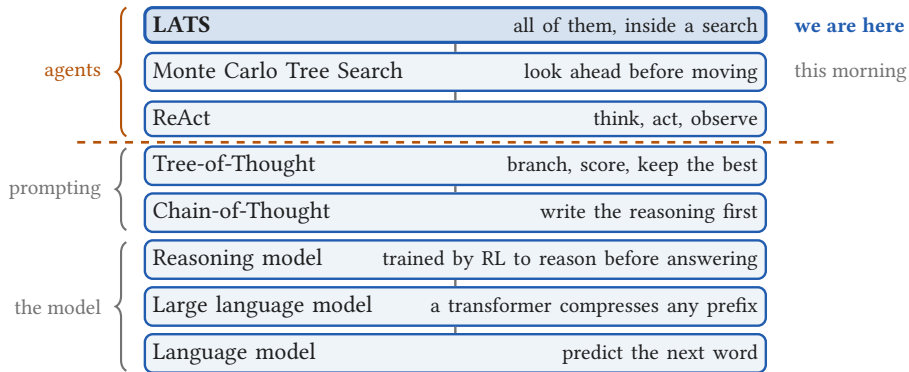


One frozen model as a planning agent

Weekend 6 · Frontier Agentic AI

Carlos Cotrini

Where LATS sits



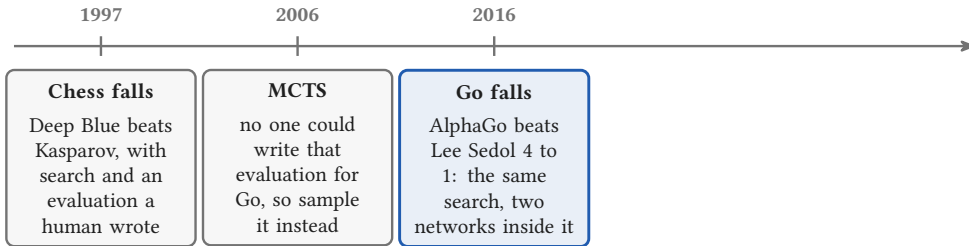
Monte Carlo Tree Search, before language models



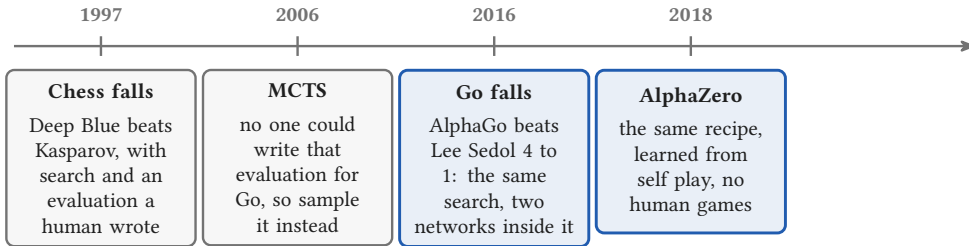
Monte Carlo Tree Search, before language models



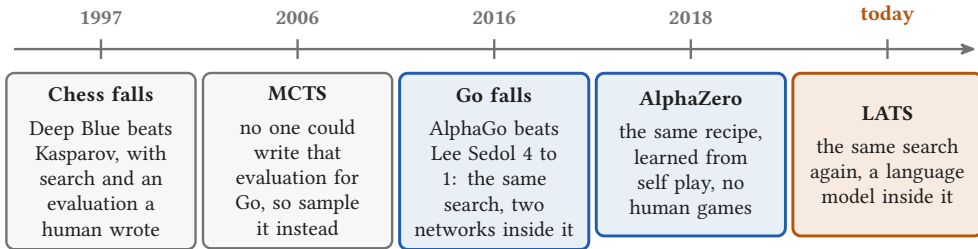
Monte Carlo Tree Search, before language models



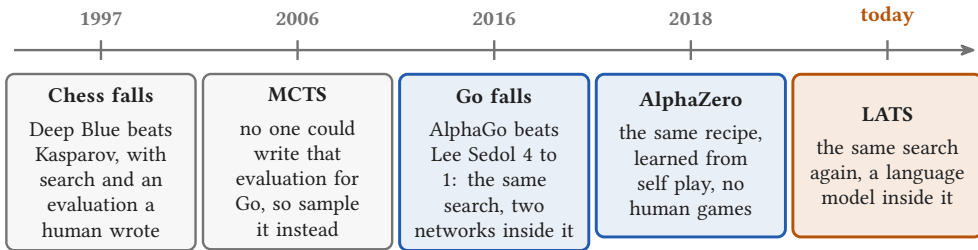
Monte Carlo Tree Search, before language models



Monte Carlo Tree Search, before language models



Monte Carlo Tree Search, before language models



The search itself barely changed across twenty years. What changed each time is what supplies the two things it needs: something to propose moves, and something to judge a position. LATS supplies both from one frozen model.

Coulom 2006; Kocsis and Szepesvari 2006; Silver et al., Nature 2016; Silver et al., Science 2018.

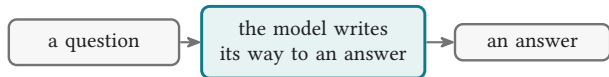


Section 1

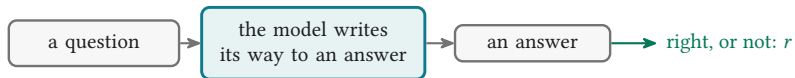
Recap: the fundamentals

1

Reinforcement learning, for reasoning

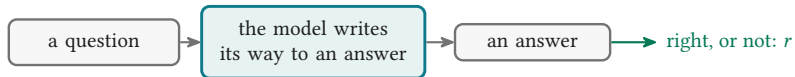


Reinforcement learning, for reasoning



Nothing scores the middle. One number arrives at the end, and it has to explain every word before it.

Reinforcement learning, for reasoning

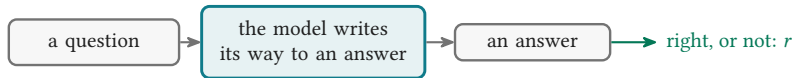


Nothing scores the middle. One number arrives at the end, and it has to explain every word before it.



Ask the same question five times. Three paid, two did not.

Reinforcement learning, for reasoning



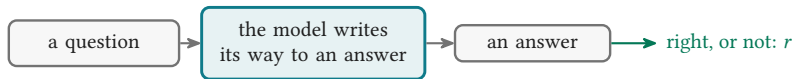
Nothing scores the middle. One number arrives at the end, and it has to explain every word before it.



Ask the same question five times. Three paid, two did not.

The group's own average is the bar to clear, so no second network has to be trained to say what a good answer was worth. Push the model toward the trajectories that beat it, away from the ones that did not.

Reinforcement learning, for reasoning



Nothing scores the middle. One number arrives at the end, and it has to explain every word before it.



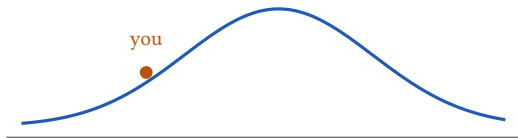
Ask the same question five times. Three paid, two did not.

The group's own average is the bar to clear, so no second network has to be trained to say what a good answer was worth. Push the model toward the trajectories that beat it, away from the ones that did not.

This is how a reasoning model is made. DeepSeek-R1-Zero is the pure reinforcement learning run, no supervised fine tuning at all: AIME 2024 goes from **15.6** to **77.9**. The chain of thought was never taught, only paid for.

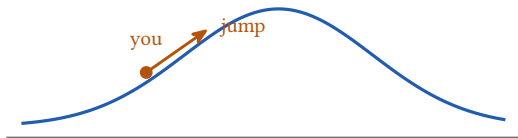
Guo et al., Nature 645, 633 to 638, 2025. Both figures are R1-Zero's.

Gradient ascent, and the grasshopper



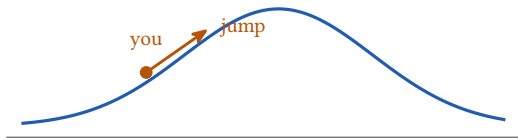
You cannot see the hill. You can feel the ground at your feet.

Gradient ascent, and the grasshopper



You cannot see the hill. You can feel the ground at your feet.

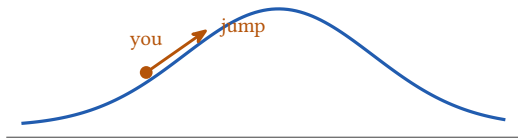
Gradient ascent, and the grasshopper



Go the way the slope points.
Go further when it is steeper.

You cannot see the hill. You can feel the ground at your feet.

Gradient ascent, and the grasshopper



You cannot see the hill. You can feel the ground at your feet.

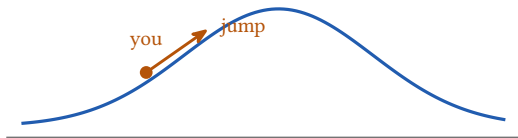
Go the way the slope points.

Go further when it is steeper.

$$\theta \leftarrow \theta + \alpha \nabla f(\theta)$$

θ where you stand, f the landscape, ∇f the slope, α how hard you jump.

Gradient ascent, and the grasshopper



You cannot see the hill. You can feel the ground at your feet.

Go the way the slope points.

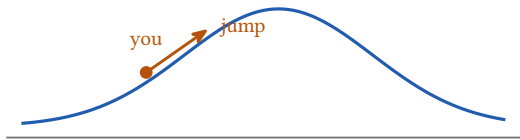
Go further when it is steeper.

$$\theta \leftarrow \theta + \alpha \nabla f(\theta)$$

θ where you stand, f the landscape, ∇f the slope, α how hard you jump.

The five levels each break one habit: a flat shelf where the slope is right but the step is too small, two hills where you walk to the nearer summit and not the better one, a narrow peak you overshoot, and rough ground where the slope you measure is contaminated. That last one is the case reinforcement learning is actually in: the gradient is estimated from sampled episodes, never measured.

Gradient ascent, and the grasshopper



You cannot see the hill. You can feel the ground at your feet.

Go the way the slope points.

Go further when it is steeper.

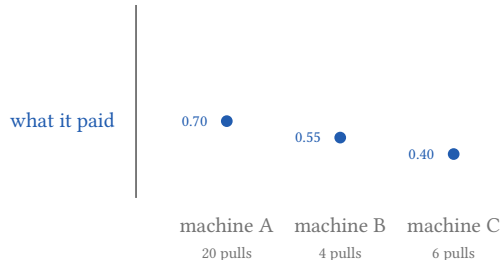
$$\theta \leftarrow \theta + \alpha \nabla f(\theta)$$

θ where you stand, f the landscape, ∇f the slope, α how hard you jump.

The five levels each break one habit: a flat shelf where the slope is right but the step is too small, two hills where you walk to the nearer summit and not the better one, a narrow peak you overshoot, and rough ground where the slope you measure is contaminated. That last one is the case reinforcement learning is actually in: the gradient is estimated from sampled episodes, never measured.

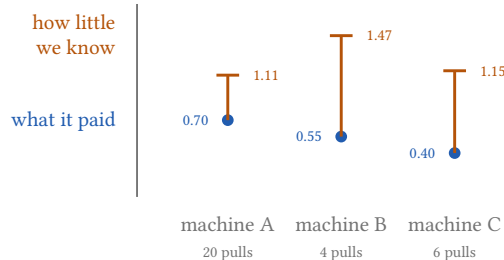
Play it: fdd-hs26.github.io/projects/grasshopper

Bandits, and the upper bound



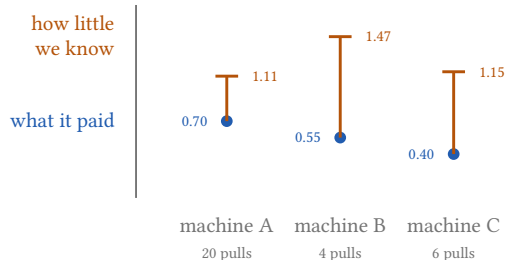
Five machines, payout probabilities hidden. You have to play and you have to learn, at once. Every pull spent finding out is a pull not spent winning.

Bandits, and the upper bound



Five machines, payout probabilities hidden. You have to play and you have to learn, at once. Every pull spent finding out is a pull not spent winning.

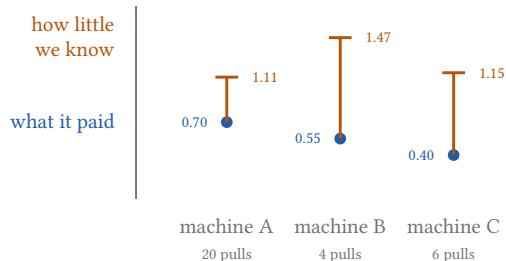
Bandits, and the upper bound



Five machines, payout probabilities hidden. You have to play and you have to learn, at once. Every pull spent finding out is a pull not spent winning.

A has the best average. **B** has been pulled four times, so almost anything is still possible there, and its *upper* bound is the highest of the three. Take **B**.

Bandits, and the upper bound

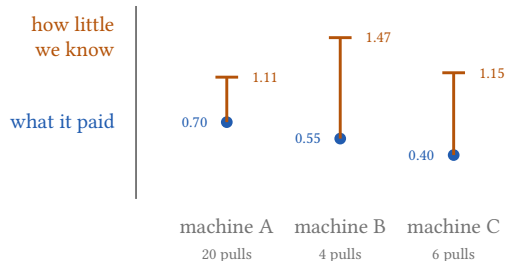


Five machines, payout probabilities hidden. You have to play and you have to learn, at once. Every pull spent finding out is a pull not spent winning.

A has the best average. **B** has been pulled four times, so almost anything is still possible there, and its *upper* bound is the highest of the three. Take **B**.

Not the highest average: the highest upper bound. A tree search picks a child the same way, and that is the rule the next section is built on.

Bandits, and the upper bound



Five machines, payout probabilities hidden. You have to play and you have to learn, at once. Every pull spent finding out is a pull not spent winning.

A has the best average. **B** has been pulled four times, so almost anything is still possible there, and its *upper* bound is the highest of the three. Take **B**.

Not the highest average: the highest upper bound. A tree search picks a child the same way, and that is the rule the next section is built on.

Play it: fdd-hs26.github.io/projects/bandits



Section 2

The LATS method

2

UCT: the critic, plus an exploration bonus

$$\text{UCT}(s) = \underbrace{V(s)}_{\text{the critic}} + \underbrace{w \sqrt{\frac{\ln N(p)}{N(s)}}}_{\text{the exploration bonus}}$$

UCT: the critic, plus an exploration bonus

$$\text{UCT}(s) = \underbrace{V(s)}_{\text{the critic}} + \underbrace{w \sqrt{\frac{\ln N(p)}{N(s)}}}_{\text{the exploration bonus}}$$

The critic

what the model thinks
this branch is worth

UCT: the critic, plus an exploration bonus

$$\text{UCT}(s) = \underbrace{V(s)}_{\text{the critic}} + \underbrace{w \sqrt{\frac{\ln N(p)}{N(s)}}}_{\text{the exploration bonus}}$$

The critic
what the model thinks
this branch is worth

+

The exploration bonus
how little we have tried it

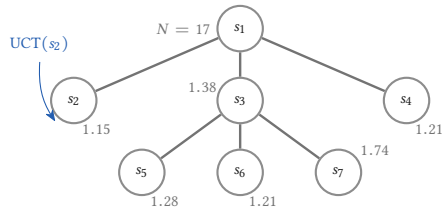
UCT: the critic, plus an exploration bonus

$$\text{UCT}(s) = \underbrace{V(s)}_{\text{the critic}} + \underbrace{w \sqrt{\frac{\ln N(p)}{N(s)}}}_{\text{the exploration bonus}}$$

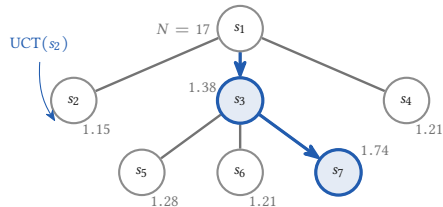


This morning's words for the same two terms: how good it looked, plus how little we know.

The same loop as this morning, with four edits



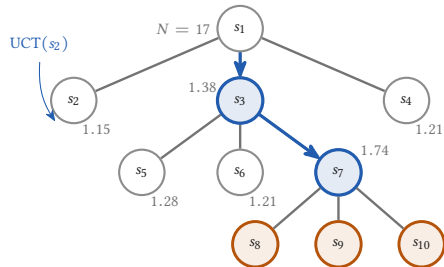
The same loop as this morning, with four edits



1 Selection

by UCT

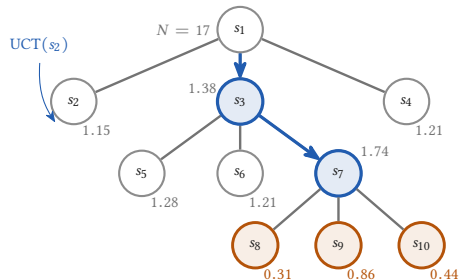
The same loop as this morning, with four edits



1 Selection by UCT

2 Expansion the *actor* proposes

The same loop as this morning, with four edits

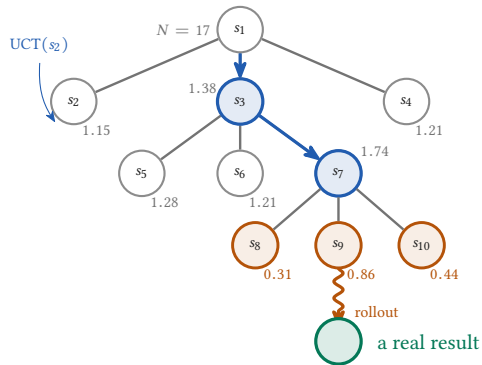


1 Selection by UCT

2 Expansion the *actor* proposes

3 Evaluation the *critic* scores

The same loop as this morning, with four edits



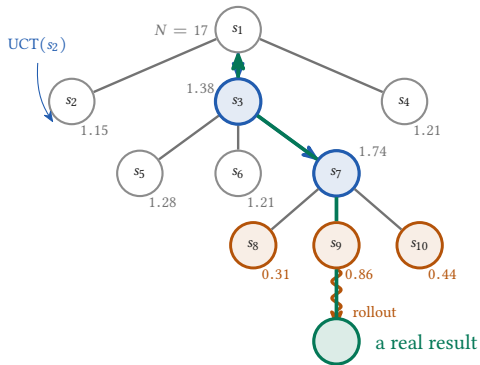
1 Selection by UCT

2 Expansion the *actor* proposes

3 Evaluation the *critic* scores

4 Simulation the *best* one, to the end

The same loop as this morning, with four edits



1 Selection by UCT

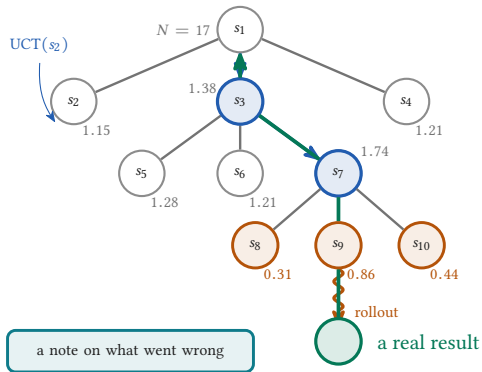
2 Expansion the *actor* proposes

3 Evaluation the *critic* scores

4 Simulation the *best* one, to the end

5 Backpropagation the *real* result, up

The same loop as this morning, with four edits



1 Selection by UCT

2 Expansion the *actor* proposes

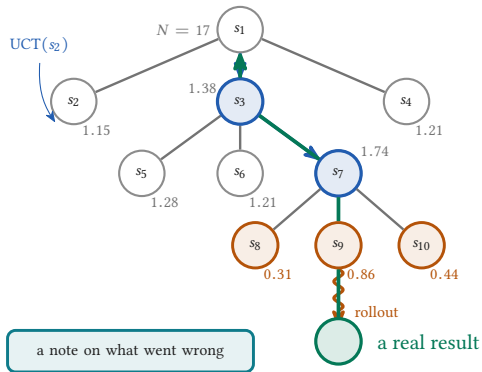
3 Evaluation the *critic* scores

4 Simulation the *best* one, to the end

5 Backpropagation the *real* result, up

6 Reflection the *feedback* model writes it

The same loop as this morning, with four edits



1 Selection by UCT

2 Expansion the *actor* proposes

3 Evaluation the *critic* scores

4 Simulation the *best* one, to the end

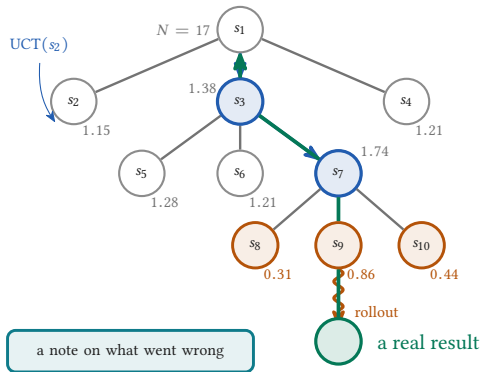
5 Backpropagation the *real* result, up

6 Reflection the *feedback* model writes it

grey: UCT, what selection compares.

amber: V from the critic, and all that is new.

The same loop as this morning, with four edits



1 Selection by UCT

2 Expansion the *actor* proposes

3 Evaluation the *critic* scores

4 Simulation the *best* one, to the end

5 Backpropagation the *real* result, up

6 Reflection the *feedback* model writes it

into the next prompt

grey: UCT, what selection compares.

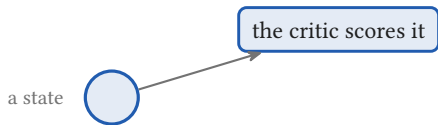
amber: V from the critic, and all that is new.

What the critic actually returns



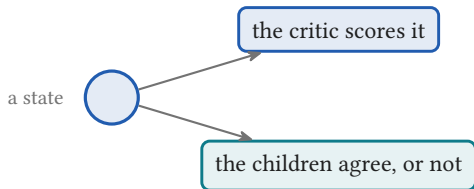
$$V(s) = \quad +$$

What the critic actually returns



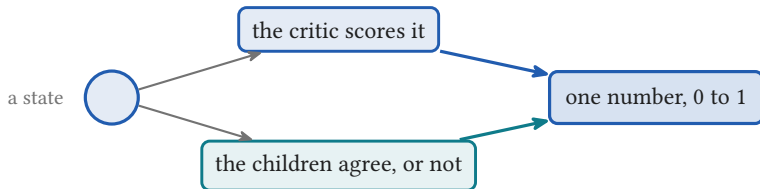
$$V(s) = \quad +$$

What the critic actually returns



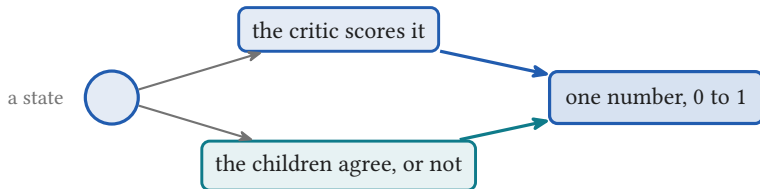
$$V(s) = \quad +$$

What the critic actually returns



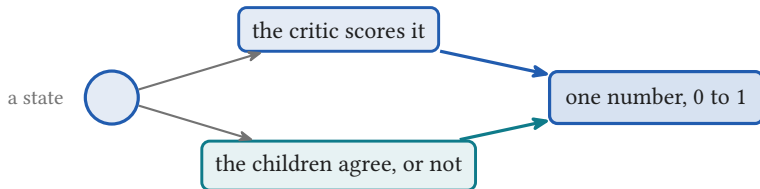
$$V(s) = \quad +$$

What the critic actually returns



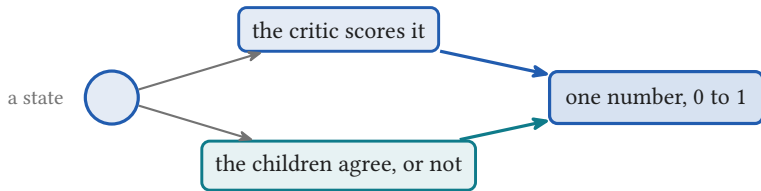
$$V(s) = \lambda LM(s) +$$

What the critic actually returns



$$V(s) = \lambda LM(s) + (1 - \lambda) SC(s)$$

What the critic actually returns



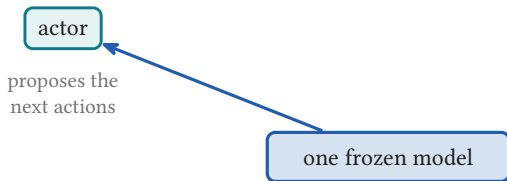
$$V(s) = \lambda LM(s) + (1 - \lambda) SC(s)$$

$\lambda = 0.5$ on question answering, 0.8 on programming and web tasks.

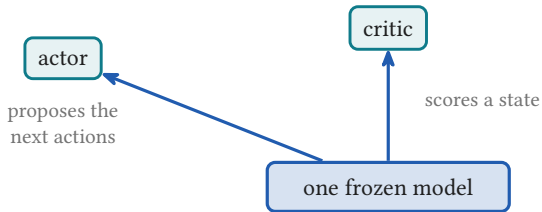
Three jobs, one set of weights

one frozen model

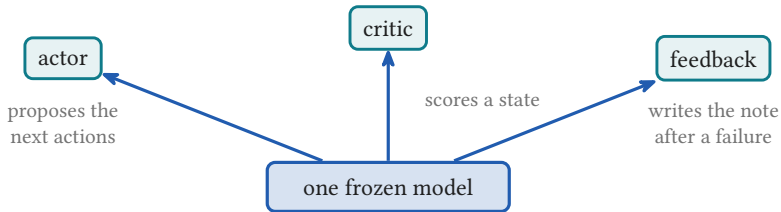
Three jobs, one set of weights



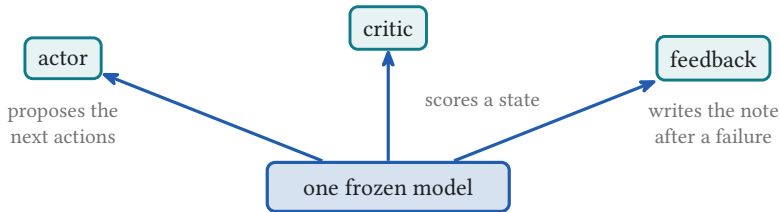
Three jobs, one set of weights



Three jobs, one set of weights



Three jobs, one set of weights



same weights every time. Only the prompt changes.